

PROYECTO FINAL UT5: Sistema de Auditoría "Kinart Chronicles"

Módulo: Lenguaje de Marcas y Sistemas de Gestión de Información

CONTEXTO DEL PROYECTO

El equipo de Quality Assurance (QA) del RPG Kinart Chronicles ha detectado partidas guardadas corruptas y posibles exploits (trampas) por parte de los jugadores. Te han entregado un archivo de guardado (save_data) en formato XML con miles de líneas de telemetría.

Tu objetivo es desarrollar un archivo auditoria.xsl que transforme este XML en un Panel de Control HTML5 (Dashboard). Este panel aplicará reglas de negocio estrictas para detectar ítems rotos, inventarios fragmentados, misiones bugeadas (softlocks) y cruzar información del equipamiento.

Se tiene que ver bien por lo que no te olvides de crear un CSS y meter todas las clases necesarias al "xsl", ten en cuenta que en varios ejercicios tendrás que meter algunas de ellas a través de atributos, pero debes tenerlas implementadas en el css también.

REQUISITOS TÉCNICOS Y PUNTUACIÓN (10 Puntos Total)

1. Arquitectura Modular (1 Punto)

Está estrictamente prohibido procesar todo el documento dentro de un único `<xsl:template match="/">`.

Debes crear la estructura HTML básica en la plantilla raíz y delegar el renderizado usando `<xsl:apply-templates>` hacia, al menos, 4 plantillas independientes (por ejemplo: metadatos, personaje, inventario, misiones).

2. Cabecera y Transformación Matemática (1.5 Puntos)

En la sección superior del Dashboard, debes mostrar la información general del jugador (Nombre, Nivel, Zona actual).

El Reto del Tiempo: El juego guarda el tiempo en segundos(seconds), usa una búsqueda para encontrar en qué línea se encuentra y no tener que hacerlo a mano, ten en cuenta que

está en inglés. Usando exclusivamente operadores matemáticos de XPath, debes calcular y mostrar por pantalla ese tiempo en el formato clásico: X Horas, Y Minutos.
(Nota: No uses los atributos hours/minutes que ya vienen, QA quiere comprobar que no ha sido hackeado calculándolo tú mismo).

3. Análisis de Personaje y Habilidades (1.5 Puntos)

Status Effects: Recorre la lista de <status_effects>. Crea una lista donde solo aparezcan los efectos que estén active="true".

Usando <xsl:choose> y <xsl:attribute>, si el efecto es "Bleeding", "Poison" o "Burning", la etiqueta debe tener la clase CSS text-danger (que al menos debe tener "color:red"). Si es "Cosiness" o "WellRested", debe tener la clase text-benefit) que al menos debe hacer que la letra sea verde.

Skills: Genera una tabla con las habilidades (<skills>). Debes usar <xsl:sort> para que aparezcan ordenadas de mayor a menor Nivel (level).

4. Auditoría de Inventario y Ejes (2.5 Puntos)

Crea la tabla principal del inventario (<inventory>). Por cada ítem:

Extrae el nombre, la rareza (si la tiene) y la cantidad (si no tiene atributo count, asume que es 1 usando un <xsl:if>) ordena la tabla según el nombre. Según la rareza debe tener una clase cuyo valor sea la rareza que tiene que cambiar de color el texto o algo más normalmente suele ser blanco verde azul violeta naranja rojo segun va incrementando la rareza, si no tiene rareza el texto que salga gris y ponga sin definir.

El atributo item_data (ej. WEAPON_Sword_Iron) contiene la categoría principal. Usa la función substring-before() para mostrar solo la primera palabra antes de la barra baja (ej. "WEAPON", "RES", "POTION"). Esto debe ir en una columna de la tabla

Si un ítem de tipo equipment tiene la etiqueta <durability>, evalúa sus atributos current y max. Si current es igual a 0, la fila entera <tr> debe tener un fondo negro y añadir el texto "[OBJETO ROTO]".

QA quiere saber si el inventario está "fragmentado". Usando el eje de hermanos anteriores, comprueba si el ítem actual tiene exactamente la misma guid que el ítem inmediatamente anterior. Si es así, añade un icono de advertencia ⚠ y el texto "Fragmentación detectada".

5. Trazabilidad de Misiones: El "Softlock Log" (2 Puntos)

QA busca un bug crítico donde las misiones se dan por completadas pero fallan en la lógica interna.

Escribe una expresión XPath que busque directamente etiquetas `<variable>` cuyo valor sea exactamente "false", pero solo si pertenecen a una misión (`<quest>`) cuyo estado sea "completed".

En un bloque llamado "Log de Errores", por cada variable fallida encontrada, debes imprimir:

El Título de la misión afectada

El ID de la misión.

El nombre de la variable corrupta.

6. El Reto del Cruce de Datos (1 Punto)

El jugador tiene ítems equipados en la etiqueta `<loadout>`.

Crea una sección "Equipamiento Actual". Recorre los `<slot>` del loadout.

Cross-referencing: Algunos slots del loadout tienen un `<guid>`. Usando ese GUID, debes buscar en todo el documento el ítem del `<inventory>` que tenga ese mismo guid, y extraer de allí su rareza (`<rarity>`).

(Pista: Deberás guardar el guid en una `<xsl:variable>` (no la hemos visto, busca información) o usar una búsqueda absoluta `/save_data/inventory/item[@guid = ...]`).

7. Cuadro de Mandos Estadístico (0.5 Puntos)

Crea un pie de página (`<footer>`) que muestre resúmenes usando funciones de agregación XPath:

Total de XP de Habilidades: Suma del atributo `@xp` de todas las `<skill>`.

Tasa de Supervivencia: Muestra la media (o porcentaje) dividiendo `<enemies_defeated>` entre `<deaths>` dentro de la etiqueta `<statistics>`.

Total de Oro Final: Oro ganado menos oro gastado (usando `<resource_stats>`).

Reto 8: Semáforo de Constantes Vitales (1.5 Puntos)

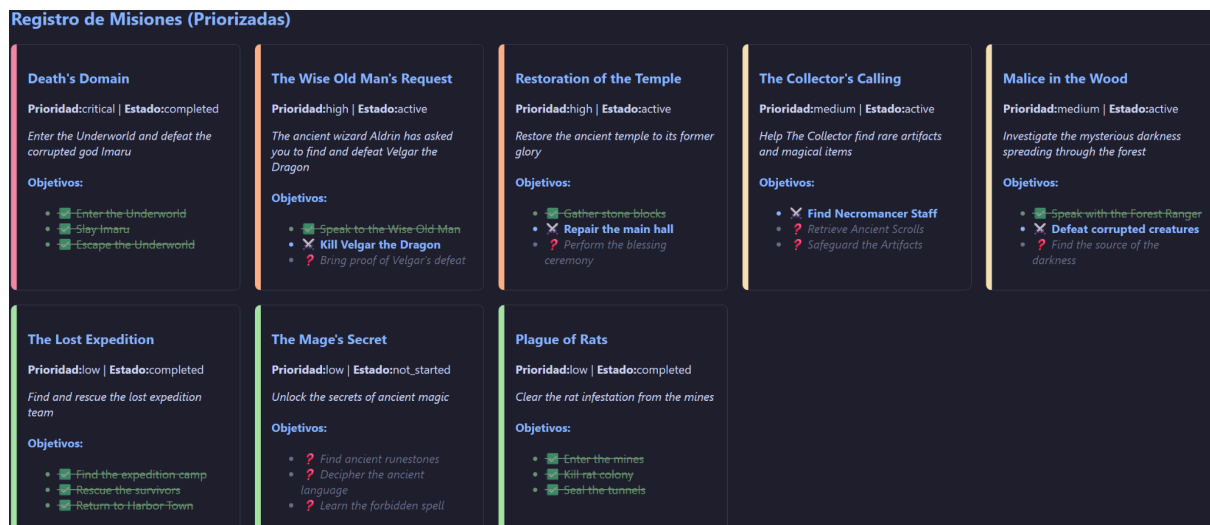
En la sección del Personaje, debes procesar la etiqueta `<vitals>`. Para cada elemento de esta etiqueta muestra su current / máximo y añade clase:

- Si está por debajo del 25%, el texto debe tener una clase CSS (ej. `vital-crítico`) que lo ponga en color rojo.
- Si está entre 25% y 60%, clase `vital-aviso` (naranja/amarillo).
- Si es mayor a 60%, clase `vital-óptimo` (verde/azul).

Esta leyenda se sigue para todos los hijos de `vitals` salvo para `toxicity` que debe ser al revés

Reto 9: Mapeo y Ordenación Lógica de Prioridades (1.5 Puntos)

Crea un panel de tarjetas para el Registro de Misiones (`<quest_log>`), cada tarjeta debe mostrar título de la misión, prioridad, estado, descripción y sus objetivos similar a la siguiente imagen



Fíjate que debes mostrar todas las misiones ordenadas por su atributo `@priority` (critical, high, medium, low).

- **El Problema:** Si usas un `<xsl:sort>` normal, ordenará alfabéticamente (C, H, L, M), lo cual es incorrecto.
- **El Reto:** Debes encontrar la manera de forzar una ordenación lógica (1º critical, 2º high, 3º medium, 4º low). (Pista: Investiga la función `translate()` de XPath).
- Además, el borde izquierdo de la "tarjeta" de cada misión debe cambiar de color según esta prioridad usando clases CSS (`priority-critical`, etc.).

Reto 10: Sistema Visual de Objetivos de Misión (1 Punto) Dentro de cada misión, debes iterar sobre sus <objectives>.

- Genera una lista . El contenido del será el texto del objetivo.
- Dependiendo del atributo @status del objetivo, la etiqueta debe recibir una clase CSS distinta:
 - completed: Texto tachado, color verde, opacidad reducida. Añade el símbolo "✅" antes del texto.
 - active / in_progress: Texto en negrita, color azul brillante. Añade "⚔️".
 - not_started: Texto en cursiva, color gris. Añade "❓".